```
#include <stdio.h>
#include <stdlib.h>

struct Node {
   char info;
   struct Node* left;
   struct Node* right;
};

struct Stack {
   char info;
   struct Node* next;
};

struct Stack st[10];
int top = -1, ssm = 0;
int i, j;
char table[9][9] = {
{'_', '+', '*', '-', '/', '^', '(', ')', '$'},
{'+', '>', '<', '>', '<', '<', '<', '>', '>'},
{'*', '>', '>', '>', '>', '<', '<', '>', '>'},
{'-', '<', '<', '>', '<', '<', '<', '>', '>'},
{'/', '>', '<', '>', '>', '<', '<', '>', '>'},
{'^', '>', '>', '>', '>', '>', '<', '>', '>'},
{'(', '<', '<', '<', '<', '<', '<', '=', ' '},
{')', '>', '>', '>', '>', '>', ' ', '>', '>'},
{'$', '<', '<', '<', '<', '<', '<', ' ', '='}
};
char s[30];

struct Node* makenode(char info, struct Node* l, struct Node* r) {
```

```
    struct Node* temp = (struct Node*)malloc(sizeof(struct Node));
    temp->info = info;
    temp->left = l;
    temp->right = r;
    return temp;
}

char check() {
    int i, j;
    for (i = 1; i < 9; i++) {
            if (table[i][0] == st[top].info) {
                break;
            }
    }
    for (j = 1; j < 9; j++) {
            if (table[0][j] == s[ssm]) {
                break;
            }
    }
    if (table[i][j] == ' ') {
            printf("Error: Invalid expression");
            getch();
            exit(0);
    }
    return table[i][j];
}

void inorder(struct Node* ptr) {
    if (ptr != NULL) {
            inorder(ptr->left);
            printf("%c ", ptr->info);
```

```
        inorder(ptr->right);
    }
}

int parse() {
    char priority;
    st[++top].info = s[ssm];
    while (1) {
        if (s[++ssm] == '$' || s[ssm] == '(' || s[ssm] == ')' || s[ssm] == '+' || s[ssm] == '*' || s[ssm]== '-'
'|| s[ssm]== '/' || s[ssm]== '^') {
            if (s[ssm] == ')' && st[top].info == '(') {
                printf("Error: Invalid expression");
                getch();
                exit(0);
            }
            if ((s[ssm] == '+' || s[ssm] == '*' || s[ssm] == '-' || s[ssm] == '/' || s[ssm] == '^') && (s[ssm +
1] == '+' || s[ssm + 1] == '*' || s[ssm + 1] == '-' || s[ssm + 1] == '/' || s[ssm + 1] == '^')) {
                printf("Error: Invalid expression");
                getch();
                exit(0);
            }
            priority = check();
            while (priority == '>') {
                st[--top].next = makenode(st[top + 1].info, st[top].next, st[top + 1].next);
                priority = check();
            }
            if (priority == '<') {
                st[++top].info = s[ssm];
            }
            else {
                if (st[top].info == '$' && !top) {
                    return 1;
```

```
                }
                if (st[top].info == '$' && top) {
                    return 0;
                }
                if (st[top].info == '(') {
                    st[--top].next = st[top + 1].next;
                }
            }
        }
        else {
            st[top].next = makenode(s[ssm], NULL, NULL);
        }
    }
}

int main() {
    printf("Enter input:");
    scanf("%s", s);
    if (parse()) {
        printf("Done\n");
        inorder(st[top].next);
    }
    else {
        printf("Not done.");
    }
    getch();
    return 0;
}
```

Output